

SYNTHETIX · SECURITY REVIEW

Synthetix

Update PR Reviews.

A security review of Update PR Reviews.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	Update PR Reviews	Guardian	November 7 – November 22, 2025	December 1, 2025

§ 00

Contents.

45 pages — overview, confidence ranking, scope, 34 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 29 findings	P. 10
	Remediation Review · 5 findings	P. 39
§ 05	Disclaimer	P. 44
§ 06	About Guardian	P. 45

§ 01

Overview.

Guardian reviewed Update PR Reviews across 2 review rounds. The final report includes 2 Critical, 4 High, 13 Medium, 10 Low, 5 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/v4-deposit-contract
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Update PR Reviews	0b97331f2078
PREPARED BY	
Guardian	
AUDIT WINDOW	
November 7 – November 22, 2025	
FINAL REPORT	
December 1, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	2	0	0	0	2
● High	4	0	0	0	4
● Medium	13	0	11	1	1
● Low	10	0	9	0	1
● Informational	5	0	4	0	1
All severities	34	0	24	1	9

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Update PR Reviews in `Synthetixio/v4-deposit-contract`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Double claiming of rewards via share transfers	Rewards	CRITICAL	RESOLVED	p. 10
H-01	Disabled Tokens Depositable On Deploy	Logical Error	HIGH	RESOLVED	p. 11
H-02	Reward Distribution Blocked by Missing Allowance	Logical Error	HIGH	RESOLVED	p. 12
M-01	Deployment Reverts With address(0) Token	Validation	MEDIUM	RESOLVED	p. 13
M-02	Role Defaults Lead To Revert	Logical Error	MEDIUM	ACKNOWLEDGED	p. 14
M-03	Missing MANAGER_ROLE Grant	Logical Error	MEDIUM	ACKNOWLEDGED	p. 15
M-04	CoW Protocol Integration Unusable	Logical Error	MEDIUM	PARTIALLY RESOLVED	p. 16
M-05	Missing MANAGER/AUTHORIZED Role Handling	Logical Error	MEDIUM	ACKNOWLEDGED	p. 17
M-06	No Readiness Checks Before Execute	Logical Error	MEDIUM	ACKNOWLEDGED	p. 18
M-07	Verification Omits Critical State	Logical Error	MEDIUM	ACKNOWLEDGED	p. 19
M-08	Operational Roles Default to TimelockController	Logical Error	MEDIUM	ACKNOWLEDGED	p. 20
M-09	Pending Withdrawal Liabilities Ignored in NAV	Logical Error	MEDIUM	ACKNOWLEDGED	p. 21
M-10	Reward Sniping via Step-Wise Jump Distribution	Logical Error	MEDIUM	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-11	Stale Fixed Withdrawal Amount Causes NAV Gaming	Logical Error	MEDIUM	ACKNOWLEDGED	p. 23
L-01	maxWithdraw Ignores Reserved Shares	Logical Error	LOW	ACKNOWLEDGED	p. 24
L-02	Disputed Withdrawal Requests Never Auto-Expire	Warning	LOW	ACKNOWLEDGED	p. 25
L-03	Deposit Path Lacks Slippage	Logical Error	LOW	ACKNOWLEDGED	p. 26
L-04	Relation Between Burnt and Output Not Enforced	Warning	LOW	ACKNOWLEDGED	p. 27
L-05	Contract Balance Check Does Not Consider Pending Rewards	Warning	LOW	ACKNOWLEDGED	p. 28
L-06	User Can Flood The Withdrawal Queue	DoS	LOW	ACKNOWLEDGED	p. 29
L-07	Reward Debt Should Round Up	Rounding	LOW	RESOLVED	p. 30
L-08	Reward Debt Can Be Scaled To Higher Decimal	Warning	LOW	ACKNOWLEDGED	p. 31
L-09	Rounding in addRewards Leaves USDT Dust	Rounding	LOW	ACKNOWLEDGED	p. 32
L-10	First Depositor Prevention Improvements	Warning	LOW	ACKNOWLEDGED	p. 33
I-01	Check Roles Script Missing Admin Role	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 34
I-02	Redundant State Checks in cancelWithdrawal	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 35
I-03	Lack Of Guardian Approval Limits in SLPVault	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 36
I-04	Duplicate User Needs To Be Prevented	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 37
I-05	Inefficient Withdrawal Request By User Search	Best Practices	INFORMATIONAL	RESOLVED	p. 38

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW					
C-01	Continuous Withdrawal DoS	DoS	CRITICAL	RESOLVED	p. 39
H-01	Incorrect Withdrawal Calculation	Logical Error	HIGH	RESOLVED	p. 40
H-02	Rewards Double-Counting At Disbursal	Logical Error	HIGH	RESOLVED	p. 41
M-01	Authorized Trader Role Not Granted	Documentation	MEDIUM	ACKNOWLEDGED	p. 42
M-02	Inconsistent Deploy and Post-deployment	Warning	MEDIUM	ACKNOWLEDGED	p. 43

CRITICAL **C-01**

RESOLVED

Double claiming of rewards via share transfers

CATEGORY	Rewards
LOCATION	https://github.com/GuardianOrg/v4-deposit-contract-team1-1762538740615/blob/master/src/SLPVault.sol#L838
REVIEW	Main Review

DESCRIPTION

The vault tracks rewards using a global accumulator (`rewardPerShare`) and a per-account checkpoint (`_rewardDebt`) that represents how much of the global rewards an account has already entitled. Deposits and withdrawals update this checkpoint appropriately and claiming rewards increase it by the amount paid. However, when vault shares are transferred between accounts, the contract does not adjust either party's `_rewardDebt` .

Consequently, shares can be moved to an address with zero (or lower) debt, that address inherits the full entitlement and can immediately claim rewards that were already claimed by the previous holder, effectively draining the whole rewards pool.

RECOMMENDATION

Update `_rewardDebt` during share transfers (`_update`) by proportionally moving debt from sender to recipient.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/27/commits/bcee1c956d3f0ae43f3686cb4af1428a4ad53b78>

HIGH **H-01**

RESOLVED

Disabled Tokens Depositable On Deploy

CATEGORY	Logical Error
LOCATION	DeploySynthetixDepositContract.s.sol: 359
REVIEW	Main Review

DESCRIPTION

Disabled-collateral flags within `getSupportedCollaterals` are ignored, so every asset listed in the mainnet defaults goes live immediately.

The configuration loop always calls `addCollateral` and `approveCowVaultRelayer` for every entry, and the implementation hard-codes `enabled: true` regardless of the provided struct. Although SNX will believe those tokens are disabled, they are actually depositable and have CoW approvals from the start.

RECOMMENDATION

Skip the `addCollateral+approval` steps whenever `enabled` is false.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/30>

HIGH **H-02**

RESOLVED

Reward Distribution Blocked by Missing Allowance

CATEGORY Logical Error
LOCATION global
REVIEW Main Review

DESCRIPTION

In SLPVault, the `addRewards` function pulls USDT via `safeTransferFrom`, which requires the USDT holder (`depositContract`) to approve the vault as a spender. The `SyntheticDepositContract` does not expose any function to set or update USDT allowances and only the token holder can grant allowance. With no on-chain path to increase the approval, the allowance remains zero and `addRewards` consistently reverts, effectively disabling reward distribution.

RECOMMENDATION

In `SyntheticDepositContract` expose an owner-only function allowing to set an approval for SLPVault.

RESOLUTION

Status: Resolved.

<https://github.com/Syntheticxio/v4-deposit-contract/pull/27/commits/1d48219039ad12f937bb2e0d7ee5b51ccbd93e0c>

MEDIUM **M-01**

RESOLVED

Deployment Reverts With address(0) Token

CATEGORY Validation

LOCATION DeploySynthetixDepositContract.s.sol, PostDeployment.s.sol, DeploySynthetixDepositContractWithTimeLock.s.sol

REVIEW Main Review

DESCRIPTION

The deployment script fails mid-execution on Sepolia (and any network with unset environment variables) due to placeholder address(0) collateral entries. The Sepolia configuration defaults SEPOLIA_USDT and SEPOLIA_WETH to address(0) when environment variables are unset, while setting non-zero allowances (type(uint256).max). During configuration, the loop correctly guards `addCollateral` calls but executes `approveCowVaultRelayer(address(0), allowance)` unconditionally when `allowance > 0`. This reverts with `TokenNotSupported()` because address(0) was never added as collateral.

Since each function call is a separate broadcast transaction, earlier operations (role grants, collateral additions) remain committed on-chain while the deployment aborts, leaving the contract half-configured and requiring manual cleanup or redeployment. The same issue exists for `setPriceFeed` and `setOracleStaleTimeoutForToken` calls which also lack `collateralToken != address(0)` guards. Note that the default address(0) also applies in PostDeployment.s.sol.

RECOMMENDATION

Validate that `collateralToken != address(0)` during deployment.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/30/commits/ad412d8a36f9adc34d0b22cf11ca562d67a95fe5>

MEDIUM **M-02**

ACKNOWLEDGED

Role Defaults Lead To Revert

CATEGORY	Logical Error
LOCATION	DeploySynthetixDepositContract.s.sol, DeploySynthetixDepositContractWithTimeLock.s.sol
REVIEW	Main Review

— DESCRIPTION

The deployment script silently defaults all operational roles to the owner address when environment variables are unset. All relay, watcher, teller, and guardian slots fall back to the owner value, which in production is intended to be a timelock contract that cannot perform fast-path operational functions like `requestWithdrawal`, `castWatcherVotes`, `disburseWithdrawals`, or emergency pauses. The default watcher configuration assigns the same owner address to all three watcher slots.

Since `grantRole` is idempotent, granting `WATCHER_ROLE` to the same address three times results in only one unique role member. When the script subsequently attempts to set `WATCHER_QUORUM = 2`, the `setWatcherQuorum` function validates that the quorum does not exceed `getRoleMemberCount(WATCHER_ROLE)` and reverts with `InvalidInput` as $2 > 1$.

This revert occurs after multiple configuration transactions have already been mined, including role grants, collateral additions, and price feed configurations, leaving the contract in a partially deployed and unusable state requiring manual recovery.

— RECOMMENDATION

Require explicit, non-zero, unique addresses for every operational role and validate that the watcher quorum does not exceed the number of unique watcher addresses before broadcasting contract calls.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Missing MANAGER_ROLE Grant

CATEGORY	Logical Error
LOCATION	DeploySynthetixDepositContract.s.sol, DeploySynthetixDepositContractWithTimeLock.s.sol
REVIEW	Main Review

— DESCRIPTION

The deployment script never grants MANAGER_ROLE to any address. After the deployer transfers OWNER_ROLE to the configured owner (intended to be a timelock) and renounces their own OWNER_ROLE, no “fast” account retains the ability to grant or revoke operational roles (RELAYER_ROLE, WATCHER_ROLE, TELLER_ROLE, GUARDIAN_ROLE, AUTHORIZED_TRADER_ROLE).

During initialization, the contract sets MANAGER_ROLE as the role admin for all five operational roles. This design allows quick operational changes without timelock delays. However, since the deployment script only grants OWNER_ROLE to the final owner and never grants MANAGER_ROLE to anyone, all operational role management becomes locked to the timelock’s execution process.

— RECOMMENDATION

Grant MANAGER_ROLE to an appropriate address before transferring OWNER_ROLE

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

PARTIALLY RESOLVED

CoW Protocol Integration Unusable

CATEGORY	Logical Error
LOCATION	DeploySynthetixDepositContract.s.sol, DeploySynthetixDepositContractWithTimeLock.s.sol
REVIEW	Main Review

DESCRIPTION

The deployment script performs "full setup" but never grants `AUTHORIZED_TRADER_ROLE` to any address, leaving the CoW Protocol integration unusable after deployment. The script configures CoW-related parameters including slippage tolerance, price feeds, and vault relayer approvals, but omits granting the role required to actually execute trades.

The ERC-1271 signature validation in `isValidSignature` recovers the EOA signer from the embedded signature and validates that the signer holds `AUTHORIZED_TRADER_ROLE`. Without this role granted to any address, all CoW swap orders will be rejected.

RECOMMENDATION

Add `AUTHORIZED_TRADER_ROLE` configuration to the deployment script.

RESOLUTION

Status: Partially resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/30/commits/f01214a9eb0f1e4a1c00b59847647cdd3a9a4455>

MEDIUM **M-05**

ACKNOWLEDGED

Missing MANAGER/AUTHORIZED Role Handling

CATEGORY	Logical Error
LOCATION	PostDeployment.s.sol
REVIEW	Main Review

DESCRIPTION

Function `executeRoleGrants` is hard-coded to grant only RELAYER/WATCHER/TELLER/GUARDIAN. No path exists to grant MANAGER_ROLE (admin for every operational role) or AUTHORIZED_TRADER_ROLE (required by `isValidSignature`). Once OWNER_ROLE lives in the timelock, role rotations and CoW trades are delayed.

RECOMMENDATION

Extend the role configuration arrays/env vars to collect manager and trader addresses, and enqueue the corresponding `grantRole` calls before the deployer relinquishes OWNER_ROLE.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

No Readiness Checks Before Execute

CATEGORY Logical Error
LOCATION PostDeployment.s.sol
REVIEW Main Review

DESCRIPTION

The script never checks `timelock.isOperationReady` or whether a matching operation was scheduled; it immediately broadcasts execute calls. If the operation ID isn't scheduled or the delay hasn't elapsed, the call reverts and is silently swallowed (try-catch) leaving the system unconfigured.

RECOMMENDATION

Before each execute, query `isOperationReady/isOperationPending` and report missing scheduling or the remaining delay.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Verification Omits Critical State

CATEGORY Logical Error
LOCATION PostDeployment.s.sol: 336
REVIEW Main Review

DESCRIPTION

Function `verifyDeployment` confirms the timelock owns OWNER_ROLE and checks a few global parameters, then merely logs counts for collaterals/roles without asserting anything. It never ensures MANAGER_ROLE or AUTHORIZED_TRADER_ROLE are granted, nor that watcher quorum is achievable (quorum <= unique watchers). Because it only compares against env vars, any change to env values between execution and verification can produce false positives.

RECOMMENDATION

Extend verification to assert the exact addresses for every required role (including manager/trader), ensure watcher quorum <= distinct watcher count, and compare on-chain config against the stored deployment plan instead of mutable env vars.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Operational Roles Default to TimelockController

CATEGORY	Logical Error
LOCATION	DeploySynthetixDepositContractWithTimelock.s.sol
REVIEW	Main Review

— DESCRIPTION

Because every relayer/watcher/teller/guardian slot defaults to the same timelockAdmin address when its env var is unset, it's easy to deploy with a single actor holding every operational role. AccessControl only stores unique members, so the three watcher entries collapse to one user; the queued setWatcherQuorum(2) call then reverts mid-run and the system launches half-configured.

— RECOMMENDATION

Require explicit, non-zero, non-timelock addresses for each operational role.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Pending Withdrawal Liabilities Ignored in NAV

CATEGORY Logical Error
LOCATION SLPVault.sol
REVIEW Main Review

DESCRIPTION

The vault does not maintain or expose global aggregates and does not subtract pending liabilities from pricing. `convertToShares` and `previewDeposit` quote mints from the gross sUSD balance (`totalAssets`) and total share supply, ignoring sUSD already owed to withdrawals. As a result, deposit pricing and on-chain views do not reflect liabilities. Users can mint against assets that are already committed to pending withdrawals and UIs cannot display a clear liability ratio.

This can mislead depositors and amplify timing-based gaming around the withdrawal queue.

RECOMMENDATION

Track and expose global aggregates and use net assets and circulating supply in `convertToShares` / `previewDeposit` to make mint pricing liability-aware.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

Reward Sniping via Step-Wise Jump Distribution

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/v4-deposit-contract-team1-1762538740615/blob/master/src/SLPVault.sol#L655
REVIEW	Main Review

— DESCRIPTION

The vault uses a step-wise jump reward accumulator - the `rewardPerShare` increases discretely when `addRewards` is executed by the TELLER. A depositor's `_rewardDebt` is set at the pre-jump level, what allows an attacker to front-run `addRewards` call with a deposit just before the reward distribution. Then, their new shares immediately capture `rewardPerShare` delta and can be claimed at once and the withdrawn process can be initiated right away.

Consequently, MEV bots can monitor and front-run the TELLER's `addRewards` calls, unfairly diluting distributions away from longer-term holders.

— RECOMMENDATION

Consider making reward eligibility time-based or epoch-based to ensure fair distribution. Alternatively, implement a cooldown mechanism where each deposit sets an eligibility timestamp and pending rewards are computed only over the portion of the user's balance that has passed the cooldown.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Stale Fixed Withdrawal Amount Causes NAV Gaming

CATEGORY Logical Error
LOCATION SLPVault.sol
REVIEW Main Review

DESCRIPTION

In SLPVault, the `createWithdrawalRequests` fixes `amountSUSD` at request creation, but the vault does not recompute the payout at disbursement. While the request is pending, shares remain in `totalSupply` and `convertToShares` computes deposit price using the gross sUSD balance, ignoring owed sUSD. If NAV per share drifts before disbursement, paying the fixed `amountSUSD` while burning the corresponding shares at the current supply transfers value between the withdrawer and all remaining holders.

Considering that $\text{fair} = \text{shares} * (\text{assets_current} / \text{supply_current})$:

- If $\text{amountSUSD} > \text{fair}$ (overpay) - remaining holders are diluted by $\text{amountSUSD} - \text{fair}$ (their per-share NAV drops).
- If $\text{amountSUSD} < \text{fair}$ (underpay) - remaining holders receive a windfall of $\text{fair} - \text{amountSUSD}$ (their per-share NAV rises).

Fixing `amountSUSD` at creation introduces unintended value transfers - NAV drift between creation and disbursement can overpay or underpay withdrawals, shifting value between remaining holders and new minters and exposing timing-based gaming and value extraction.

RECOMMENDATION

Recompute the sUSD payout at disbursement as `convertToAssets(shares)` using the current NAV, optionally enforcing a slippage tolerance.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

maxWithdraw Ignores Reserved Shares

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/v4-deposit-contract-team1-1762538740615/blob/master/src/SLPVault.sol#L240
REVIEW	Main Review

DESCRIPTION

The `maxWithdraw` function uses the full share balance (`balanceOf(owner)`) as the basis for the withdrawable amount. It ignores the user's reserved shares, which back an existing withdrawal request. Also, it ignores the fact that a user with an active withdrawal request (`_userActiveWithdrawalId[owner] != 0`) cannot create another request.

Because withdrawals in SLPVault are asynchronous and go through a separate request lifecycle, a user can only withdraw:

- at most their unreserved (available) shares,
- only if they do not already have an active request.

Despite this, `maxWithdraw` returns the assets corresponding to the entire balance, even when most or all of those shares are reserved and the user is blocked from initiating additional withdrawals. This breaks the vault expectation that `maxWithdraw` reflects what the user can actually withdraw.

RECOMMENDATION

Update `maxWithdraw` to be request and reservation aware, so it reflects what a user can actually withdraw.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Disputed Withdrawal Requests Never Auto-Expire

CATEGORY Warning
LOCATION SLPVault.sol
REVIEW Main Review

DESCRIPTION

Function `disputeWithdrawals` moves a request from Approved to Disputed, but `_isWithdrawalExpired` only checks the Approved and Validated states. Once a request is disputed it is immune to the expiry timer forever, so `_reservedShares[user]` and `_userActiveWithdrawalId` remain locked until a privileged role intervenes. A single dispute can therefore freeze a user's shares indefinitely, enabling DoS against the vault.

RECOMMENDATION

Clearly document this risk.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Deposit Path Lacks Slippage

CATEGORY Logical Error
LOCATION SLPVault.sol
REVIEW Main Review

— DESCRIPTION

In SLPVault, the deposit function mints at the spot rate and lacks a `minSharesOut` slippage check. Between quote and execution, the mint rate can worsen if price-per-share changes due to asymmetric state changes (e.g. mispriced withdrawal disbursements) or direct sUSD donations.

Because the function does not validate the minted amount against a user-supplied tolerable value, a depositor can receive fewer shares than expected with no way to revert based on price movement.

— RECOMMENDATION

Add a slippage check that verifies the computed number of shares is within a user-specified minimum before proceeding with the deposit.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Relation Between Burnt and Output Not Enforced

CATEGORY Warning
LOCATION SLPVault.sol
REVIEW Main Review

DESCRIPTION

Function `createWithdrawalRequests` lets the relayer provide any (shares, amountSUSD, amountUSDT) tuple as long as the user owns at least that many shares and the vault currently holds the requested tokens. There is no check that amountSUSD corresponds to `convertToAssets(shares)` (plus rewards)

During `disburseWithdrawals`, the teller blindly burns shares and sends the prefilled amountSUSD/amountUSDT. In the most extreme case, it is possible to burn a token holder's 1 wei of shares and pay them the entire vault's sUSD balance.

RECOMMENDATION

Consider enforcing the share<>asset conversion on chain during the withdrawal flow. Otherwise, ensure off-chain checks are appropriately performed.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Contract Balance Check Does Not Consider Pending Rewards

CATEGORY Warning
LOCATION SLPVault.s.sol
REVIEW Main Review

— DESCRIPTION

When a relayer queues a withdrawal it must only show that the vault currently holds enough balance in the contracts for the requested `amountUSD`: `if (contractBalanceUSDT < amountUSD)`

However, this does not consider the user's pending rewards. Consequently, users can claim rewards between request creation and disbursement to DoS the `disburseWithdrawal` queue.

— RECOMMENDATION

Consider pending rewards in the balance check, although this would be a soft-check as pending rewards can increase by disbursement time. For a stricter check, explicitly reserve USDT at request creation and include the withdrawer's pending rewards in that reservation.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

User Can Flood The Withdrawal Queue

CATEGORY DoS
LOCATION SLPVault.sol
REVIEW Main Review

DESCRIPTION

Any address can deposit the minimum 1 sUSD, then request a withdrawal through off-chain flow. Because `createWithdrawalRequests` processes every user individually and enforces no per-batch cap or fee, an attacker who spins up hundreds of funded wallets can force the RELAYER/WATCHER/TELLER roles to handle thousands of tiny requests on mainnet.

Each batch becomes prohibitively expensive, and the backlog delays honest withdrawals long enough that they hit `withdrawalExpiryTimeout` and expire, leaving users stuck in `Status.Expired` even though the problem is pure spam. This also grieves the Relayer as the contract is expensive and cost grows with more withdrawals.

RECOMMENDATION

Enforce a minimum withdrawal amount and consider a withdrawal fee.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

Reward Debt Should Round Up

CATEGORY Rounding
LOCATION SLPVault.sol
REVIEW Main Review

DESCRIPTION

Reward debt is calculating using `(shares * rewardPerShare) / 1e18` which rounds down. However, this rounds in the user's favor since the smaller the debt, the more rewards they can claim. For safety, the reward debt calculation should round up.

RECOMMENDATION

Round up when calculating reward debt.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/27/commits/53255caa718414e5d6d9fcb00beb6a175fafa156>

LOW **L-08**

ACKNOWLEDGED

Reward Debt Can Be Scaled To Higher Decimal

CATEGORY Warning
LOCATION SLPVault.sol
REVIEW Main Review

— DESCRIPTION

Reward per share is 6 decimals and is used in reward and reward debt calculations. It may be prudent to scale this value to higher precision (e.g. 18 decimals) to avoid unintended rounding scenarios with rewards/debt.

— RECOMMENDATION

Consider magnifying reward per share and reward debt to a higher decimal precision before payout (ensure on payout precision goes back to 6 decimals for USDT)

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Rounding in addRewards Leaves USDT Dust

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/v4-deposit-contract-team1-1762538740615/blob/master/src/SLPVault.sol#L655
REVIEW	Main Review

— DESCRIPTION

`addRewards` updates `rewardPerShare` using floor division, so each distribution leaves an unallocated remainder that is not accumulated and included in the next reward update. While per-user rounding in `pendingRewards` / `claimRewards` rolls into future claims, this distribution-level remainder becomes permanent USDT dust that accumulates in the contract and creates a drift between USDT received and USDT ever claimable.

— RECOMMENDATION

To minimize adding complexity, consider simply document this behavior. If a fix is implemented, add an only-owner sweep function that transfers only the USDT surplus above the explicitly tracked unclaimed rewards, and any non-core stray tokens. An alternative would be accumulating the remainder from each `addRewards` call in a dedicated variable and include it in the next reward update so that all USDT transferred into the vault via `addRewards` is eventually claimable by users.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

First Depositor Prevention Improvements

CATEGORY Warning
LOCATION SLPVault.sol
REVIEW Main Review

— DESCRIPTION

The first deposit can be withdrawn below the minimum deposit amount, hence the initial supply is not locked and the classic "dead shares" fix to first-depositor is not implemented.

— RECOMMENDATION

Consider having the protocol lock up the initial deposit and prevent the shares from going below that initial deposit on withdrawal so that supply is locked. Furthermore, consider increasing the first deposit amount.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Check Roles Script Missing Admin Role

CATEGORY Warning
LOCATION CheckRoles.s.sol
REVIEW Main Review

DESCRIPTION

The script claims to check "all role assignments" but it never queries DEFAULT_ADMIN_ROLE (0x00). This isn't a large gap since it is never explicitly granted but should be noted.

RECOMMENDATION

Consider including the DEFAULT_ADMIN_ROLE in the check and any future roles.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Redundant State Checks in cancelWithdrawal

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/v4-deposit-contract-team1-1762538740615/blob/master/src/SLPVault.sol#L410 https://github.com/GuardianOrg/v4-deposit-contract-team1-1762538740615/blob/master/src/SynthetixDepositContract.sol#L501
REVIEW	Main Review

DESCRIPTION

The cancelWithdrawal functions in both SLPVault and SynthetixDepositContract effectively allow user-initiated cancellation only from a single status (SLPVault: `Status.Approved`, SynthetixDepositContract: `Status.Requested`) and only when the request is not expired. After the early expiry check, the current logic redundantly excludes all other states (`Validated`, `Disputed`, and all final states: `Disbursed`, `Denied`, `Cancelled`, `Expired`), which is functionally equivalent to a single explicit state precondition and adds unnecessary complexity that can obscure intent and increase maintenance risk.

RECOMMENDATION

In SLPVault, replace the compound condition with a simple precondition requiring `Status.Approved` after the expiry check and in SynthetixDepositContract, require `Status.Requested`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Lack Of Guardian Approval Limits in SLPVault

CATEGORY Warning
LOCATION SLPVault.sol: 640
REVIEW Main Review

DESCRIPTION

Function `resolveDisputedWithdrawal` in `SynthetixDepositContract` enforces per-token guardian approval limits, preventing a single guardian from green-lighting payouts above a configured cap. The analogous function in `SLPVault` simply flips a disputed request back to `Validated` with no size checks.

A compromised guardian can therefore approve a withdrawal that empties the entire vault, bypassing the usual watcher quorum, while the deposit contract would have blocked that action.

RECOMMENDATION

Consider adding guardian approval limits as in the `SynthetixDepositContract`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Duplicate User Needs To Be Prevented

CATEGORY Warning
LOCATION Off-chain
REVIEW Main Review

— DESCRIPTION

`createWithdrawalRequests` iterates a batch supplied by the relayer, and if a user occurs twice, the second occurrence would trigger `_userActiveWithdrawalId[user] != 0` and the function reverts, aborting the entire transaction.

If a malicious caller is able to force the relayer to include their address twice will be able to brick the whole batch, preventing any other users' withdrawals from being queued.

— RECOMMENDATION

Ensure the Relayer has safety checks to ensure a user cannot be included twice.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

RESOLVED

Inefficient Withdrawal Request By User Search

CATEGORY Best Practices
LOCATION SLPVault.sol: 326
REVIEW Main Review

DESCRIPTION

Currently `getWithdrawalRequestsByUser` loops through all existent withdrawal requests and validates matches with the passed `_user`. Although this functions according to interface spec, it would be more efficient to having a mapping for (user => userRequestIds). Currently, if a user has sparse requests, the caller to `getWithdrawalRequestsByUser` has to keep bumping `_offset` and rescanning large gaps just to find entries.

RECOMMENDATION

Consider a mapping to allow for more efficient by-user querying.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/27/commits/0d14eea34b4815b3aaf8eb974cf98575e5c54d33>

CRITICAL **C-01**

RESOLVED

Continuous Withdrawal DoS

CATEGORY DoS
LOCATION SLPVault.sol: 424
REVIEW Remediation Review

DESCRIPTION

Function `requestWithdrawal` no longer has access control, hence an attacker can pass a `_user` and create a withdrawal. Because a user can only have one withdrawal at a time, the attacker can trigger a withdrawal for just 1 wei and continuously block the user from actually withdrawing their funds.

RECOMMENDATION

Ensure the `_user` is `msg.sender`.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/1ffac1bca9d35fe7544dfc53f124d4ce>

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/27/commits/5d01748f85eba513b500e9172988a576bccd2454>

HIGH **H-01**

RESOLVED

Incorrect Withdrawal Calculation

CATEGORY Logical Error
LOCATION SLPVault.sol: 442
REVIEW Remediation Review

DESCRIPTION

The `requestWithdrawal` function incorrectly uses `previewWithdraw(_shares)` to calculate the sUSD amount users should receive, however function `previewWithdraw` should accept assets as the parameter. Consequently, users can receive a fraction of the intended withdrawal.

For example:

- Share price = 2 sUSD / share
- User withdraws 100 shares
- Should receive: 100 shares * 2 = 200 sUSD
- Actually receives: 100 shares * 0.5 = 50 sUSD, loss of 150 sUSD

RECOMMENDATION

Calculate `_amountSUSD` with `uint256 _amountSUSD = convertToAssets(_shares);`

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/3a1914c73759b227adc746d2b71644d8>

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/27/commits/5d01748f85eba513b500e9172988a576bccd2454>

HIGH **H-02**

RESOLVED

Rewards Double-Counting At Disbursal

CATEGORY Logical Error
LOCATION SLPVault.sol
REVIEW Remediation Review

DESCRIPTION

Function `disburseWithdrawals` adds `pendingRewards()` to the already stored `req.amountUSDT` from request time, causing double-counting since `pendingRewards()` calculates total accumulated, not delta since request. Consequently, users will receive double the rewards.

RECOMMENDATION

Consider calculating amountUSDT at disbursal time or update the reward debt at request time.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/0fa366737c5b6930050a754b4e6f1ef5>

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/v4-deposit-contract/pull/27/commits/59de01d642a5e9e5ea41d57e5a06d4d870d086a6>

MEDIUM **M-01**

ACKNOWLEDGED

Authorized Trader Role Not Granted

CATEGORY Documentation
LOCATION DeploySynthetixDepositContractWithTimeLock.s.sol
REVIEW Remediation Review

DESCRIPTION

The DeploySynthetixDepositContractWithTimeLock script does not schedule a grant `AUTHORIZED_TRADER_ROLE`, leaving the CoW Protocol integration unusable after deployment

RECOMMENDATION

Schedule a grantRole operation for the `AUTHORIZED_TRADER_ROLE` in the script.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Inconsistent Deploy and Post-deployment

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

PostDeployment.s.sol is designed to execute timelock operations scheduled by DeploySynthetixDepositContractWithTimeLock.s.sol, but uses fundamentally different collateral configurations that will cause all execution attempts to fail. The timelock identifies scheduled operations by hashing the complete calldata. When DeploySynthetixDepositContractWithTimeLock.s.sol schedules addCollateral(token, config) with specific parameters (e.g. USDT with userMaximum: 100_000_000 * 1e6), it creates a unique hash.

PostDeployment.s.sol attempts to execute with completely different config values (USDT with userMaximum: 0), generating a different hash. The timelock will reject these as unscheduled operations. The limits between the same collaterals across the two scripts are different, as well the which tokens are enabled are different. Additionally, PostDeployment's use of 0 for all limit fields causes the buildCollateralSetup function to default userMaximum to `type(uint256).max`, creating unlimited deposit configurations if the script somehow succeeded

RECOMMENDATION

Remove duplicate collateral configuration logic from PostDeployment.s.sol and make it use the same configuration source as DeploySynthetixDepositContractWithTimeLock.s.sol

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.